全国大学生嵌入式芯片与系统设计竞赛

FPGA 创新设计赛道

amd 自主选题赛道初级组

基于 *Zynq-7Z100* 的
光电器件缺陷检测系统

面向 *PIN* 光电探测器的嵌入式视觉检测

技　术　文　档

开发平台　　*Vivado / Vitis 2026.1*

目标器件　　*Zynq-7Z100*

修订日期　　*2026* 年 *11* 月 *3* 日

摘要

随着工业自动化与精密制造的发展，产品表面缺陷检测已成为生产质量控制的重要环节。在电子制造、半导体加工及精密器件生产中，自动化视觉检测能够辅助识别表面异常，为产品筛选与工艺改进提供依据。光电探测器是实现光电信号转换的重要器件，广泛应用于光通信、工业测量和医疗分析等领域，其制造质量受到持续关注。因此，面向光电探测器开展表面缺陷检测研究，具有明确的工程应用价值。

光电探测器表面缺陷存在尺寸差异大、形态多样、部分缺陷与背景纹理相近等特点，给准确识别与定位带来困难。人工目视检测依赖操作人员经验，面对大量图像时，检测效率与判断一致性容易受到影响；基于固定阈值和人工设计特征的传统视觉方法，则需要针对光照、纹理及缺陷形态变化进行适配。近年来，深度学习为工业表面缺陷的特征提取与自动识别提供了新的技术途径，但微小缺陷、有限样本及模型计算开销仍是实际应用需要解决的问题。

针对上述需求，本项目构建基于 *YOLOv8n* 与 *Zynq-7Z100* 的嵌入式检测系统，实现 *PIN* 光电探测器九类表面缺陷的定位与分类。系统采用 *PS* 与 *PL* 协同架构，*ARM* 端完成输入量化、计算图调度及检测后处理，*PL* 端通过 *12* 路并行整数计算承担卷积和最大池化任务。针对计算资源与存储带宽限制，采用 *INT8* 量化、双权重打包、*DSP* 级联和片上特征复用，减少参数存储与重复数据搬运；通过 *ACP* 共享内存连接软硬件计算环节，并结合局部控制寄存、分簇布局与时序优化，实现检测模型在可编程逻辑上的部署。

在使用方式上，*PC* 端按照连接检查、图像导入、开始检测、结果查看与导出的顺序组织操作，支持多图选择、文件夹导入和队列检测。原图与标注图并排展示，缺陷框与详情表同步呈现类别、置信度和位置，便于操作人员核查异常区域。界面同时提供暂停与继续、历史查询、检测图及 *JSON* 报告导出，并在连接异常或请求失败时给出状态提示，将原本分散的通信、检测和记录操作整合到图形界面，减少命令行输入，降低使用门槛。

完成 *200* 轮训练后，在包含 *800* 张图像、*2295* 个缺陷目标的固定 *640×640* 输入离线验证中，浮点模型的 *mAP@0.5* 检测精度为 *97.23%*，*INT8* 模型为 *97.04%*，量化后下降 *0.19* 个百分点；*mAP@0.5:0.95* 分别为 *79.67%*和 *77.89%*，下降 *1.78* 个百分点。在 *PL* 工作频率为 *100 MHz*、采用 *JTAG* 启动的 *SCOPED* 缓存配置时，实板连续请求耗时中位数为 *362.42 ms*，*P95* 为 *364.63 ms*，批次吞吐率为 *2.77 FPS*。上述请求包含连接、图像传输、板端推理与响应校验，不含界面展示等待；两幅图像交替输入的原始检测头与整数参考保持一致。项目将模型训练量化、软硬件推理和可视化结果管理贯通，为光电探测器表面缺陷的嵌入式检测提供了可核查的实现方案与验证依据。

关键词：缺陷检测；*FPGA* 加速；可视化交互

目　录

摘要.....1
第一部分　作品概述.....4
1.1 功能与特性.....4
1.2 应用需求与相关工作.....4
1.3 主要技术特点.....5
1.4 主要性能指标.....5
1.5 主要创新点.....6
1.6 设计流程.....6
第二部分　系统组成及功能说明.....7
2.1 整体介绍.....7
2.1.1 软硬件划分依据.....9
2.1.2 接口设计与数据交接.....9
2.2 硬件系统设计.....9
2.2.1 硬件平台与系统接口.....9
2.2.2 *PL* 加速架构与数据通路.....10
2.2.3 *INT8* 计算与片上存储设计.....11
2.2.4 控制握手与时序实现.....11
2.3 软件系统设计.....12
2.3.1 模型训练与部署工具链.....12
2.3.2 *PS* 运行时与计算图调度.....12
2.3.3 *PS-PL* 驱动与缓存管理.....13
2.3.4 检测后处理与结果返回.....13
2.3.5 上位机界面与通信协议.....14
第三部分　完成情况及性能参数.....14
3.1 系统完成情况.....14
3.2 工程实现成果.....15
3.2.1 硬件集成与部署产物.....15
3.2.2 板端推理与通信软件.....15
3.2.3 上位机交互与结果管理.....15
3.2.4 模型与配套资料.....16
3.3 性能测试与结果分析.....16
3.3.1 离线检测精度与量化损失.....16
3.3.2 最终版实板数值一致性.....16
3.3.3 最终整机资源与时序.....17

3.3.4 实板处理耗时与连续请求吞吐.....18
3.3.5 完成情况与综合验证.....18
3.4 优化过程与前后对比.....18
3.4.1 基线选择与瓶颈定位.....18
3.4.2 首次跑通基线与最终版对照.....18
3.4.3 同一设计的物理优化与时序收敛.....19
3.4.4 未直接采用的方案与经验.....19
第四部分 大模型协作与技能包提炼.....20
4.1 协作分工与工作流程.....20
4.2 典型协作记录与纠错过程.....20
4.2.1 先纠正目标模型，再讨论部署成功.....20
4.2.2 根据收益决定训练何时收尾.....20
4.2.3 放弃没有收益的循环改写.....21
4.2.4 诊断通过不等于原版故障消失.....21
4.3 *协作经验提炼的技能包*.....21
4.4 通用 *PYNQ* 诊断技能.....22
4.4.1 用途与通用设计.....22
4.4.2 换题目与换板卡的复用验证.....22
4.4.3 交付材料与评审入口.....22
第五部分 复现说明.....23
5.1 准备工程包和工具.....23
5.2 解压并建立 Python 环境.....23
5.3 核对工程文件和模型输入.....24
5.4 从 RTL 重建硬件并运行组件仿真.....24
5.5 编译固件并生成启动镜像.....24
5.6 上板启动并运行桌面检测.....25
5.7 复现 300 张样例的检测输出.....26
5.8 保存复现结果.....26
第六部分 总结.....27
6.1 可扩展之处.....27
6.2 心得体会.....27
第七部分 参考文献.....28

第一部分　作品概述

1.1 功能与特性

本作品面向 *PIN* 光电探测器芯片，实现破损、裂痕、击穿、薄膜破损、电极缺失、异物、有机物污染、部分缺失和冗余九类表面缺陷的定位与分类，输出缺陷类别、置信度及矩形框坐标。

系统配套可视化上位机，集成连接检查、图片选择、文件夹导入和队列检测功能。连接检查用于排查网络、通信协议和模型配置等问题。其中 *SHA-256* 充当模型文件和类别表的识别器，用于发现两端版本不一致，帮助定位由此造成的连接失败。界面并排展示原图与检测图，直观标注缺陷位置，并提供检测详情、暂停与继续、历史查询及结果导出功能，方便逐图核查和结果追溯。用户通过按钮即可完成日常检测，减少命令行操作，降低使用门槛。

系统采用 *YOLOv8n*，输入为 *640×640 RGB* 图像，经等比例缩放和 *INT8* 量化后在板端执行。*PL* 承担卷积等主要计算，*PS* 负责任务调度与后处理。



图 *1-1* 系统检测效果示例

1.2 应用需求与相关工作

PIN 光电探测器广泛用于光通信接收、激光测距和医疗成像。芯片在切割、镀膜和电极制作中可能产生裂痕、薄膜破损、电极缺失或污染。这类缺陷尺寸小、形态差异大，目前多依赖显微镜下的人工目检，效率受限于检验员经验和疲劳程度，细裂痕与背景纹理相近时容易漏检。

本系统可部署在检测工位旁，与显微相机配合，对芯片图像进行在线或离线初筛，给出带置信度的缺陷框，供人工复检和质量统计使用。推理在边缘端 *FPGA* 上完成，图像不必上传服务器，适合对数据保密和响应时延有要求的产线。

此外，数据治理、训练、*INT8* 量化、*PS/PL* 部署和逐层验证的方法可用于其他缺陷检测任务。迁移到 *PCB*、晶圆或不同器件时，需要重新核对数据划分、类别与输入约定、算子覆盖、量化精度和板端性能，再决定哪些模块可以直接复用。

本项目采用 *YOLOv8n* 作为检测基础，主要工作集中在适配 *Z100* 的 *INT8* 执行、可布线的并行计算结构、模型到板端的数值核对，以及可直接操作的检测界面。不同研究使用的平台、数据集和输入条件不同，因此不据此比较 *FPS* 或宣称算法精度领先。项目贡献以自身优化记录和完整应用链路为依据。

1.3 主要技术特点

（*1*）并行计算与布线协同设计。针对计算单元增多后连线复杂、难以落地的问题，将布线优化与加速架构一起设计，使多路并行计算结构在 *100 MHz* 下完成整机布线并通过时序检查，让计算能力真正落实到板卡。

（*2*）从模型训练到检测应用完整贯通。将训练、量化、板端部署和上位机操作整合为一套系统，统一模型、类别和坐标规则。图像导入后可直接完成检测、原图标注与结果保存，减少各环节之间的人工转换。

（*3*）板端独立完成九类缺陷检测。将完整检测网络和结果筛选部署到嵌入式板卡，直接返回缺陷类别、位置及置信度；*PC* 端负责操作和展示，检测计算无需依赖 *PC* 显卡或云端服务，便于在检测工位旁部署。

（*4*）量化部署兼顾精度与可核验性。将模型转换为适合板端计算的 *INT8* 形式，离线检测精度保持 *97.04%*，较浮点模型仅下降 *0.19* 个百分点；同时核对板端与参考结果，兼顾缺陷识别能力和部署结果的一致性。

（*5*）把检测和复核集中到可视化界面。连接检查、批量选图、队列控制、原图与检测图对照、缺陷详情及历史导出均可通过界面完成。异常时提供排查线索，检测后可逐图核查，降低操作门槛并方便结果追溯。

1.4 主要性能指标

表 *1-1* 主要性能指标

指标	数值	说明
检测类别	*9* 类	*PIN* 光电器件表面缺陷；输入 *640×640 RGB* 图像
INT8 检测精度	*97.04% / 77.89%*	分别为 *mAP@0.5* 和 *mAP@0.5:0.95*；*800* 张验证图离线评估

指标	数值	说明
量化精度损失	*0.19* / *1.78* 个百分点	对应上述两项精度，相比同一浮点模型均下降不足 *2* 个百分点
满载计算密度	约 *61.34%*	满载时每拍 *1536* 次 *INT8* 乘法，占 *1252* 个 *DSP* 按双 *INT8* 乘法配置折算容量的比例；为理论设计指标
推理耗时	约 *265.48 ms*	板端内部推理耗时中位数，包含输入处理、图计算及解码
吞吐率	约 *2.68 FPS*	连续检测批次实测，包含批次额外开销

1.5 主要创新点

（*1*）面向并行计算的布线设计。针对大量计算单元同时工作带来的长连线和信号分发压力，将权重、偏置和控制信号分组复制并就近寄存，缩短关键数据传递路径。最终整机在 *100 MHz* 下通过布线后的时序检查，使并行计算结构能够实际落到板卡上运行。

（*2*）贯通模型到实际检测的整合设计。将模型训练、*INT8* 量化、板端软硬件推理和上位机展示接入同一套检测流程，统一模型、类别和图像坐标规则。从导入原图到返回缺陷位置、保存结果，各环节可以直接衔接，形成可运行、可核查的完整系统。

（*3*）兼顾故障定位与结果复核的可视化界面。把连接检查、图片队列、原图与检测图对照、缺陷详情和历史导出集中在同一界面。连接异常时提供排查线索，检测后可逐图查看并保存结果，让使用者通过按钮完成日常操作，降低部署和复核门槛。

1.6 设计流程

设计流程分为以下八步：

（*1*） **数据治理**：按来源组划分训练集、验证集和测试集。

（*2*） **训练与选模**：以验证集 *mAP@0.5:0.95* 作为选模指标。

（*3*） ***INT8* 量化与打包**：生成板端部署所需的模型包。

（*4*） **整数参考与 *C* 后端**：建立整数计算参考，完成 *C* 后端实现。

（*5*） ***RTL* 设计与验证**：完成硬件设计，进行逐拍等价验证。

（*6*） **路径检查与时序收敛**：开展综合阶段的路径普查与时序优化。

（*7*） **整核仿真验证**：通过 *xsim golden* 验证核对完整计算核的输出。

（*8*） **整板集成与测试**：完成整板集成，并开展上板测试。

每一步都以上一步的输出作为参照，出现差异就在本层定位，不跨层下结论。



图 *1-2* 设计流程图

第二部分　系统组成及功能说明

2.1 整体介绍

系统由 *PC* 端、*PS* 端和 *PL* 端三部分组成，整体框图见图 *2-1*。

***PC* 端**负责数据集管理、模型训练、*INT8* 量化和模型包生成，也运行桌面客户端，用于发送图像和显示结果。

***PS* 端**为双核 *Cortex-A9*，以裸机方式运行。它负责图像预处理、按模型包中的事件表调度整网、执行上采样和拼接等非卷积算子，并对检测头做解码与 *NMS*。

PL* 端是自行设计的加速核。PS* 通过 *AXI GP0* 写入控制寄存器，加速核通过 *ACP* 从共享内存读取参数和输入特征，并把输出写回共享内存，完成后经 *IRQ_F2P* 通知 *PS*。

上位机连接板卡时，会核对双方使用的模型文件和缺陷类别顺序。若发现版本不一致，界面会给出提示，帮助判断连接失败是否由模型配置不匹配引起。

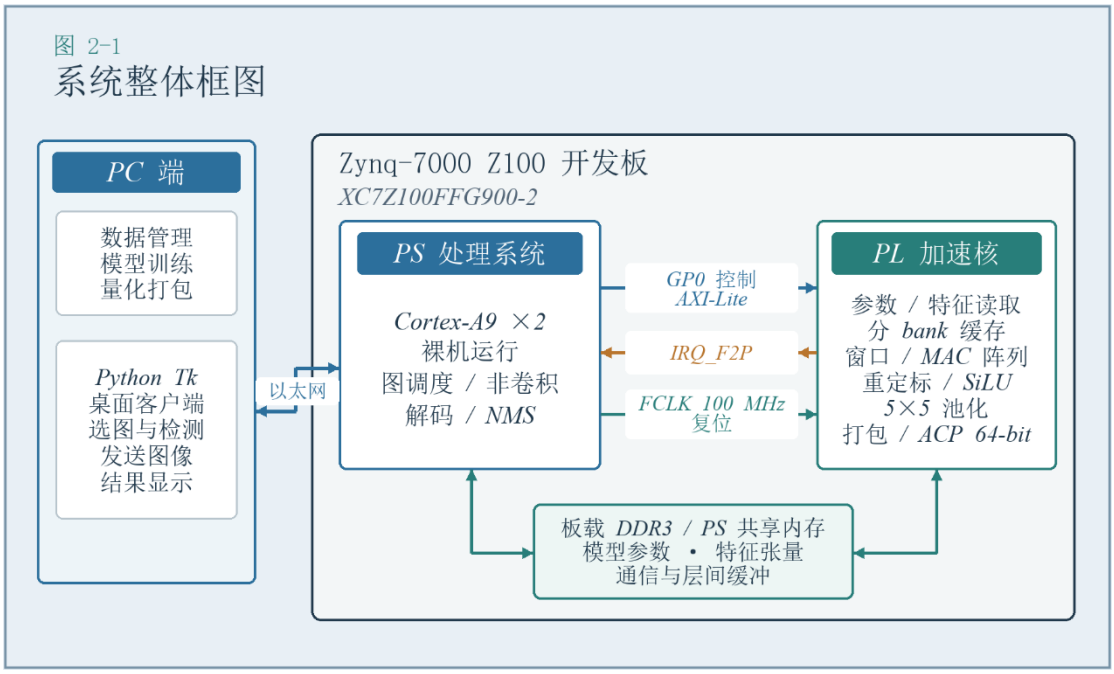


图 *2-1* 系统整体框图

表 *2-1* 软件与硬件的功能划分

划分	组成	主要功能
软件	模型训练与部署工具	整理缺陷数据，训练九类检测模型，完成 *INT8* 量化并生成板端模型文件。
	PC 可视化上位机	检查连接，导入图片或文件夹，管理检测队列，对照显示原图与缺陷标注，查询历史并导出结果。
	PS 板端推理程序	接收图像，完成输入量化，安排模型各层计算，整理数据并筛选检测框，返回类别、置信度和位置。
	通信与连接诊断	通过以太网传输图像和结果，核对模型及类别配置，对连接失败、超时和结果异常给出提示。
硬件	*Zynq-7Z100* 开发平台	集成 *ARM* 处理器和 *FPGA*，承载板端软件与加速逻辑，连接上位机及外部存储。
	FPGA 计算加速模块	并行执行卷积乘加，完成数值调整、激活、最大池化及结果打包。
	存储与数据接口	*DDR* 保存模型参数与中间结果；片上缓存复用计算数据；*ACP* 连接处理器与 *FPGA* 的数据访问。

注：*PC* 为上位机；*PS* 为 *Zynq* 片上的 *ARM* 处理系统；*PL* 为 *FPGA* 可编程逻辑。

2.1.1 软硬件划分依据

卷积与最大池化需要对大量特征数据重复执行相似计算，适合利用 *PL* 并行处理、片上缓存和数据复用。计算图调度、张量视图、*DFL* 解码及 *NMS* 包含较多条件判断和候选筛选，放在 *PS* 端便于实现和调整。*PC* 端承担文件管理、图像缩放、显示与结果导出，使板端资源集中于推理，同时方便操作者批量检测和复核。

划分时同时考虑运算量、数据搬运、实现成本和修改频率。并非所有操作放入 *PL* 都会更快：若新增硬件带来的数据整理、传输或布线开销超过计算收益，应保留软件实现。实际取舍以分项计时、数值一致性和整机时序检查共同判断。

2.1.2 接口设计与数据交接

表 *2-2* 系统接口及完成条件

接口	传递内容	完成与异常处理
PC—PS：*TCP*	*HELLO*、*640×640 RGB* 图像、检测结果	先核对协议和模型；按长度收发；超时或异常不显示成功。
PS—PL：*GP0 / AXI-Lite*	任务参数、地址、控制与状态	*PS* 提交任务并读取状态；用任务代次区分前后请求。
PL—DDR：*64* 位 *AXI / ACP*	模型参数、特征及输出	按缓存策略维护数据；硬件写回确认后才发布结果。
PL—PS：*IRQ_F2P*	完成或异常通知	通知后核查状态；异常时排空已接受事务再退出。

2.2 硬件系统设计

2.2.1 硬件平台与系统接口

系统采用正点原子 *Zynq-7000 Z100* 开发板，器件为 *XC7Z100FFG900-2*，集成双核 *ARM Cortex-A9* 处理系统和 *FPGA* 可编程逻辑。器件提供 *277400* 个 *LUT*、*554800* 个寄存器、*755* 个 *BRAM36* 及 *2020* 个 *DSP*，板载 *DDR* 用于模型参数、特征张量及通信缓冲。

PS 通过 *AXI GP0* 连接加速核的 *AXI-Lite* 控制接口，配置任务描述符并读取执行状态；*PL* 数据端采用 *64* 位 *AXI* 接口，经 *ACP* 访问 *PS* 内存系统。*PL* 计算时钟为 *100 MHz*，任务完成或异常状态通过中断和状态寄存器交由 *PS* 处理。上位机与板卡通过以太网交换图像和检测结果，系统连接见图 *2-2*。

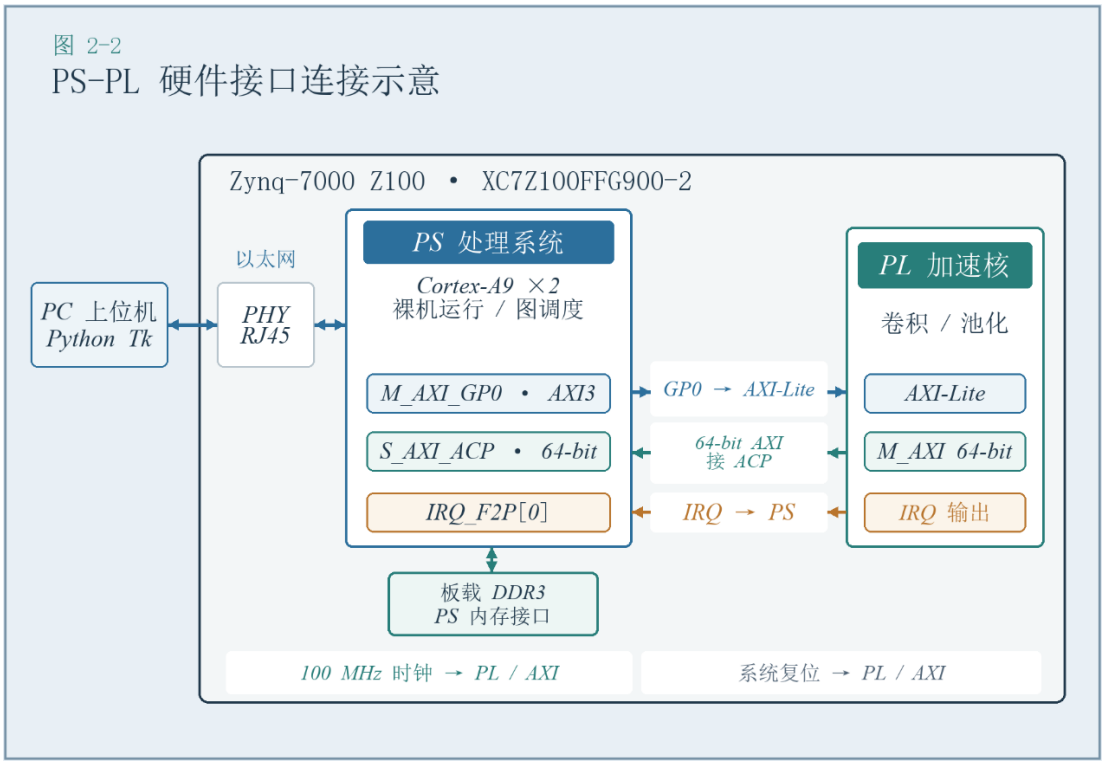


图 2-2 最终版硬件平台与 PS-PL 接口

2.2.2 PL 加速架构与数据通路

PL 采用原生 SystemVerilog 实现卷积与 SPPF 最大池化加速。ACP 接口封装模块为 z100_native_conv_pool_acp_v1，内部连接任务控制、参数读取、特征缓存、窗口生成、并行乘加、整数重定标、激活查表及输出打包模块。计算阵列采用 S12-L32-4tap 结构，处理 12 个空间位置、32 个输出通道和 4 个输入通道抽头。

卷积任务按计算图给出的参数与特征地址执行，窗口数据进入乘加阵列后，依次完成累加、重定标、激活和输出打包，再经 AXI 写回共享内存。池化支路复用特征缓存和访问接口，执行 SPPF 中的 5×5 最大池化。片上复用减少同一任务内的重复读取，层间结果仍按计算图进行内存读写，模块关系见图 2-3。

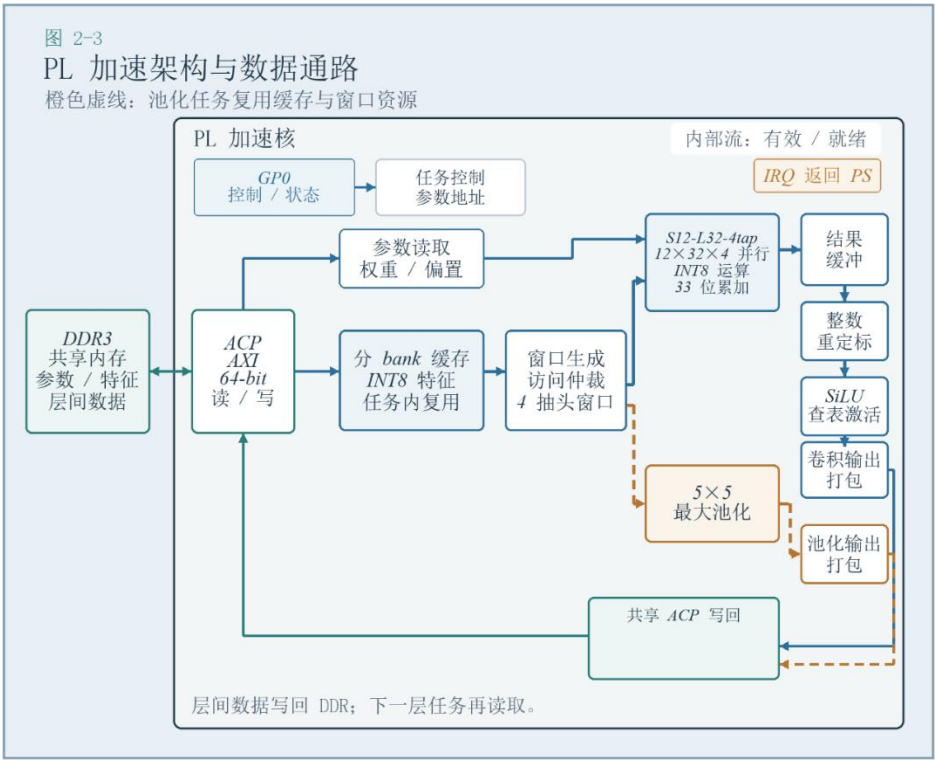


图 *2-3 PL* 加速架构与数据通路

2.2.3 *INT8* 计算与片上存储设计

计算阵列采用双权重打包与四级 *DSP* 级联。对共享激活值 a 的两个权重，按 $p = w_h \times 2^{17} + w_l$ 构成打包操作数，四个抽头的乘积在 *DSP* 级联通路内求和，再通过符号修正提取两组整数部分和。权重范围限定为*−127* 至 *127*，避免超出打包表达范围；部分和进入 *33* 位反馈累加器，随后按模型参数完成整数重定标和激活处理。

特征张量在片上按 *bank* 组织，通过窗口地址共享与访问仲裁供卷积和池化读取；权重与偏置在当前计算组内复用。结果缓冲与输出打包器协调计算端和 *AXI* 写回端速率，使用有效/就绪握手及流量控制处理下游阻塞，避免未完成结果被提前覆盖。

2.2.4 控制握手与时序实现

阵列控制采用局部寄存和分簇复制，降低权重、偏置与控制信号的扇出；累加反馈尽量保留在 *DSP* 内部，关键模块边界设置寄存缓冲以缩短组合路径。控制器跟踪任务代次及已接受的 *AXI* 事务，发生取消或错误时等待必要的读写响应排空，防止未完成事务与后续任务混用。

最终整机实现使用 *1252* 个 *DSP*、*95260* 个 *LUT* 和 *287.5* 个 *BRAM36* 等效块。在 *10 ns* 时钟约束下，布线后 *WNS* 为*+0.005 ns*、*WHS* 为*+0.045 ns*、*TNS* 为 *0*，建立时间与保持时间失败端点均为 *0*。上述结果对应最终 *S12* 与 *ACP* 设计。

2.3 软件系统设计

2.3.1 模型训练与部署工具链

软件由 *PC* 离线训练与部署工具、*PS* 裸机运行时和 *PC* 图形客户端组成。离线工具基于 *Ultralytics YOLOv8* 完成九类 *PIN* 缺陷模型训练、权重选择、*INT8* 量化和模型包生成；*PS* 执行板端推理与后处理；客户端管理图片、提交检测请求并展示结果，软件关系见图 *2-4*。

数据按已知原图来源组划分为 *6400* 张训练图、*800* 张验证图和 *800* 张测试图。*YOLOv8n* 完成 *200* 轮训练，依据验证集 *mAP@0.5:0.95* 选择第 *200* 轮权重。量化使用训练集图像校准，生成配套 *INT8* 参数、计算图、重定标与激活数据及有序类别表，并与模型包身份关联。

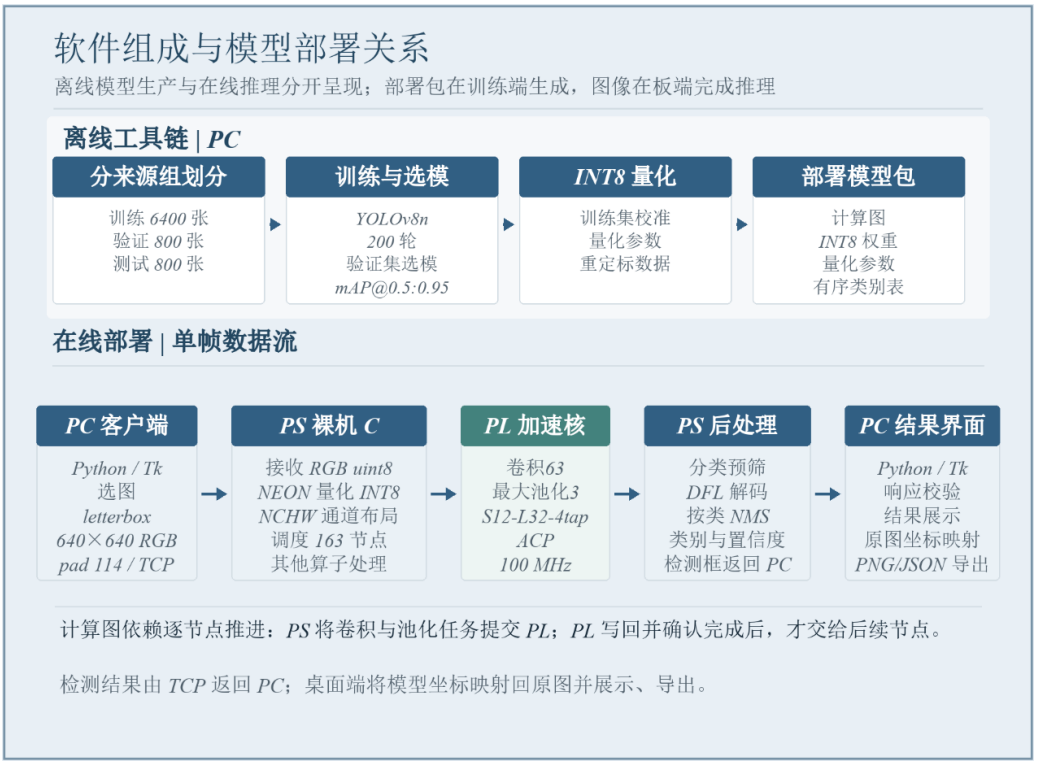


图 *2-4* 软件组成与模型部署关系

2.3.2 *PS* 运行时与计算图调度

PS 端以裸机 *C* 程序运行网络服务、输入处理、计算图执行和检测后处理。上位机先完成等比例 *letterbox* 缩放及 *114* 填充，发送 *640×640* 的 *RGB uint8* 图像。*PS* 通过 *ARM NEON* 完成输入量化与通道分离，得到 *INT8 NCHW* 张量，再整理为计算核心使用的存储布局；板端不重复进行 *letterbox* 缩放。

调度器管理计算图中 *163* 个节点的数据依赖与缓冲区生命周期，对 *63* 个卷积节点和 *3* 个最大池化节点下发 *PL* 任务，并完成其余节点对应的视图管理、数据整理与软件

运算。上采样、拼接及切片等操作结合张量视图和按需物化处理，支持的重排路径采用 *NEON*、连续访存和等价整数变换，减少重复地址计算。

2.3.3 *PS-PL* 驱动与缓存管理

驱动通过 *GP0* 控制寄存器提交任务、读取状态并确认执行结果，*PL* 通过 *ACP* 读取参数和特征、写回输出。软件将源缓冲区的读租约、目标缓冲区的写入预留和任务代次关联，只有在硬件完成并确认写回后才发布结果，供后续节点使用。

缓存维护按 *ACP* 启动状态和地址范围策略执行，对满足一致性条件的受控区域减少冗余维护，其余情况保留必要的清理、失效和内存屏障。超时或硬件错误进入受控退出与事务排空流程，不将部分结果标记为有效，也不以局部复位直接丢弃已接受的总线事务。

2.3.4 检测后处理与结果返回

后处理接收 *80×80*、*40×40* 和 *20×20* 三个尺度的 *INT8* 检测头，共 *8400* 个网格、*613200* 个输出字节。每个网格包含 *64* 个边框分布通道和 *9* 个类别通道。*PS* 先执行分类预筛，再对候选位置进行 *DFL* 解码和框坐标计算，最后执行按类别 *NMS*，输出缺陷类别、置信度及位置。

当前检测置信度阈值为 *0.25*，*NMS IoU* 阈值为 *0.45*，最多输出 *300* 个目标。板端返回模型输入坐标系下的归一化矩形框，上位机依据 *letterbox* 映射关系将其投影回原图。图 *2-5* 给出了板端单帧处理流程。

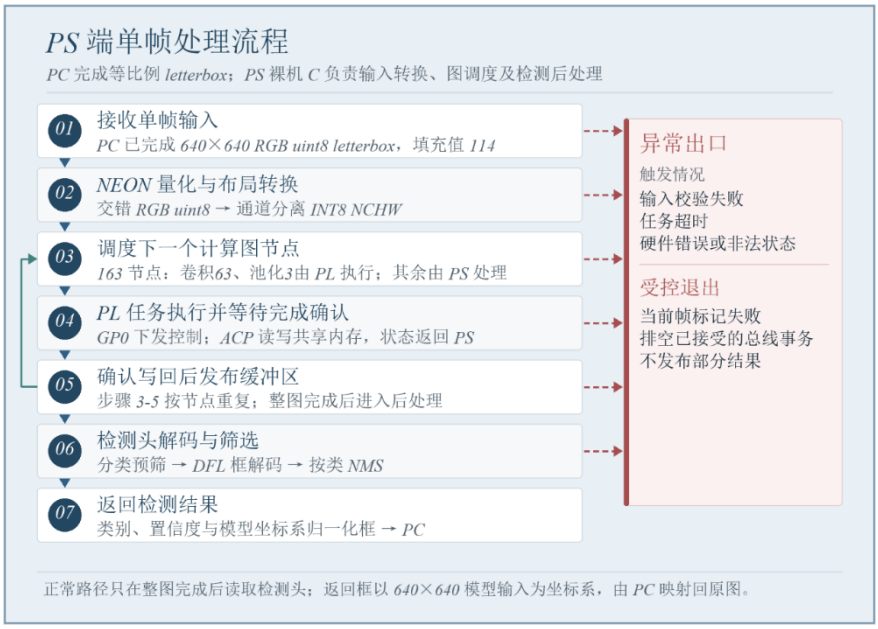


图 *2-5 PS* 端单帧处理流程

2.3.5 上位机界面与通信协议

上位机采用 *Python* 图形界面，按“检查连接—选择图片—开始检测—查看与导出”组织操作。支持多图选择、文件夹导入和队列检测，并排显示原图与检测图，在图像中标注缺陷位置，并通过详情表显示类别、置信度和坐标；提供展示间隔设置、暂停与继续、停止后续任务、历史查询和检测图及 *JSON* 报告导出，减少命令行操作。

客户端通过 *TCP* 与板端服务通信，连接时执行 *HELLO* 握手，核对协议、模型包 *SHA-256*、有序类别表身份及九类 *PIN* 输入约定，再按固定长度发送 *RGB* 数据。板端负责模型推理，上位机负责请求管理、响应校验、坐标映射与可视化。连接失败、超时或数据异常均通过状态提示呈现，避免将未确认的请求结果显示为检测完成。

第三部分　完成情况及性能参数

3.1 系统完成情况

本项目已完成基于 *Zynq-7Z100* 的 *PIN* 光电探测器九类表面缺陷检测原型，实现“上位机导入图像—网络提交—板端推理—结果返回—可视化与导出”的完整链路。当前版本采用第 *200* 轮 *YOLOv8n* 模型，输入为 *640×640 RGB* 图像，*PL* 采用 *S12-L32-4tap* 原生 *RTL* 架构，在 *100 MHz* 下运行，通过 *ACP* 与 *PS* 共享数据。

系统以现成开发板为硬件载体，完成 *PL* 加速逻辑、*PS* 裸机运行时及 *Python* 上位机集成。*PC* 完成图像等比例缩放与填充，*PS* 完成输入量化、计算图调度和检测后处理，*PL* 承担卷积及 *SPPF* 最大池化等计算，最终输出类别、置信度和缺陷框。系统实物连接与运行状态见图 *3-1*。

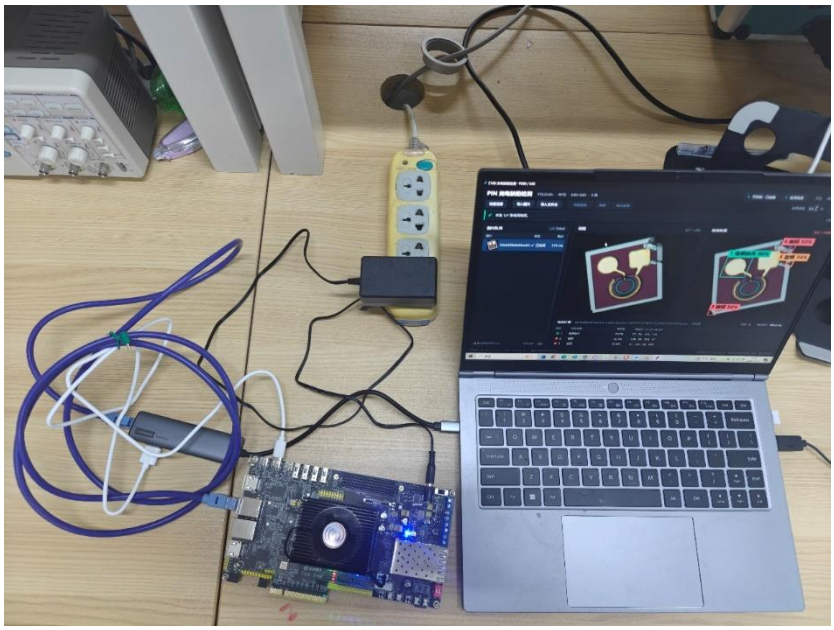


图 *3-1*　系统实物连接与运行状态

3.2 工程实现成果

3.2.1 硬件集成与部署产物

完成 *XC7Z100FFG900-2* 平台的 *PS-PL* 集成，采用 *GP0* 及 *AXI-Lite* 配置任务，*64* 位 *AXI* 数据通路经 *ACP* 读取参数与特征并写回结果。*PL* 集成特征缓存、窗口生成、*S12* 并行乘加、整数重定标、*SiLU* 查表、池化及输出打包，已形成配套 *bitstream*、*XSA* 和板端 *ELF* 文件。最终整机布线、时序与资源结果见 *3.3.3*。

3.2.2 板端推理与通信软件

PS 裸机程序完成 *TCP* 服务、*NEON* 输入量化及布局转换、*163* 节点计算图调度、缓冲区所有权管理和错误状态处理。卷积与池化任务由 *PL* 执行，*PS* 负责视图管理、非卷积数据整理及三尺度检测头的分类预筛、*DFL* 解码和 *NMS*。当前置信度阈值为 *0.25*、*NMS IoU* 阈值为 *0.45*，最多返回 *300* 个目标。

连接阶段通过 *HELLO* 核对通信协议、模型包 *SHA-256* 及有序类别表身份，减少模型与类别不匹配造成的错误。单帧结果返回模型坐标系下的归一化框，上位机再映射至原始图像，完成板端计算与图形显示之间的数据衔接。

3.2.3 上位机交互与结果管理

上位机已实现连接检查、多图选择、文件夹导入及队列检测。原图与检测图并排展示，检测框与详情表同步呈现类别、置信度和坐标；支持展示间隔设置、暂停与继续、停止后续任务、历史查询以及标注图 *PNG* 和 *JSON* 结果导出。用户通过按钮即可完成检测与核查，减少命令行操作，降低使用门槛，并保留可追溯的检测记录。

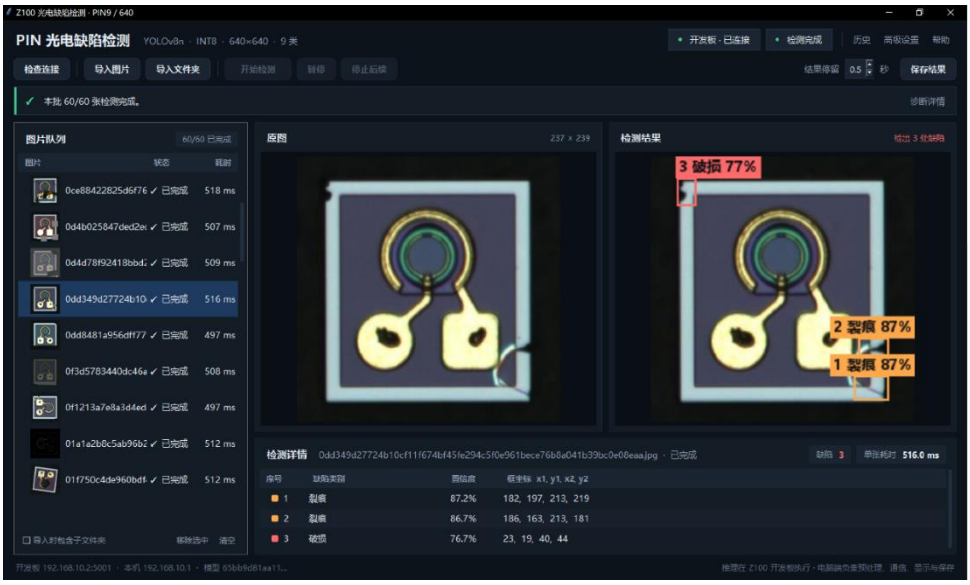


图 *3-2* 上位机检测界面与结果管理

3.2.4 模型与配套资料

完成九类 *PIN* 缺陷 *YOLOv8n* 的 *200* 轮训练，以冻结验证集 *mAP@0.5:0.95* 选择第 *200* 轮权重，并形成浮点权重、*INT8* 模型包、计算图、量化参数与类别表。项目同时保存训练与量化报告、硬件实现报告、最终版实板采集结果和源码，为模型部署、结果核对及后续维护提供依据。

3.3 性能测试与结果分析

本节分别列出固定 *640×640* 输入的离线精度、最终版实板数值一致性、整机实现结果和请求性能。离线验证、板端内部计时与主机请求计时各自使用对应统计范围。

3.3.1 离线检测精度与量化损失

采用同一第 *200* 轮模型及固定 *640×640* 预处理，对 *800* 张验证图、*2295* 个标注目标开展浮点与 *INT8* 离线评估。浮点 *mAP@0.5* 和 *mAP@0.5:0.95* 分别为 *97.23%*和 *79.67%*，*INT8* 分别为 *97.04%*和 *77.89%*，下降 *0.19* 和 *1.78* 个百分点，均满足量化损失不超过 *2* 个百分点的离线门槛。逐类结果见表 *3-1*。

表 *3-1* 固定 *640×640* 输入的离线验证精度

类别	目标数	浮点 *AP50*/%	浮点 *AP50:95*/%	*INT8 AP50*/%	*INT8 AP50:95*/%
破损	*680*	*98.63*	*81.10*	*98.16*	*79.33*
裂痕	*392*	*92.66*	*71.85*	*92.65*	*70.63*
击穿	*84*	*99.50*	*97.34*	*99.50*	*97.43*
薄膜破损	*361*	*97.97*	*77.49*	*96.94*	*76.30*
电极缺失	*141*	*99.50*	*86.54*	*99.50*	*87.51*
异物	*100*	*98.50*	*88.11*	*98.50*	*85.39*
有机物污染	*353*	*89.38*	*74.71*	*89.23*	*73.00*
部分缺失	*78*	*99.50*	*73.14*	*99.50*	*66.05*
冗余	*106*	*99.48*	*66.77*	*99.41*	*65.35*
总体 *mAP*	*2295*	*97.23*	*79.67*	*97.04*	*77.89*

3.3.2 最终版实板数值一致性

最终版完成同一冻结 *RGB* 图像的大量连续主机请求，均返回经验证的板端结果，与参考响应一致，无失败或未知结果，且未发生重试。在连续请求前后及 *A*、*B* 两种输入交替的检查中，共采集多份完整 *INT8* 检测头，每份 *613200* 字节均与各自整数参考逐字节一致，相关中断保护检查通过。

上述结果验证了所检查输入下的整网数值执行与跨任务数据交接。重复输入和少量图像的逐字节一致性检查，不能替代完整数据集实板精度、长时间稳定性或独立生产批次泛化验证。

3.3.3 最终整机资源与时序

表 *3-2* 中，满载计算密度按阵列每拍 *1536* 次 *INT8* 乘法，除以整机 *1252* 个 *DSP* 按双 *INT8* 乘法配置折算的每拍 *2504* 次乘法容量计算，约为 *61.34%*。该指标描述阵列满载时的设计能力，不代表实际检测过程中的平均算力发挥率。*PL* 工作频率为 *100 MHz*，*WNS/TNS* 为*+0.005/0 ns*，*WHS/THS* 为*+0.045/0 ns*，建立时间、保持时间及脉宽检查均通过。*238792* 条网络全部完成布线，布线错误和关键 *DRC* 均为 *0*。

表 *3-2* 最终整机资源占用与满载计算密度

资源类型	占用率	满载计算密度 （按双乘法配置）
DSP	*1252 / 2020（61.98%）*	按双 *INT8* 乘法配置 计算的满载设计能力约 *61.34%*
LUT	*95260 / 277400（34.34%）*	
寄存器	*98764 / 554800（17.80%）*	
BRAM36 等效块	*287.5 / 755（38.08%）*	



图 *3-3* 最终整机实现报告

3.3.4 实板处理耗时与连续请求吞吐

在最终 *epoch200*、*S12*、*100 MHz*、*ACP* 版本上，向板端顺序提交同一冻结 *640×640 RGB* 图像，进行 *30* 次请求。每次请求包含连接、*HELLO*、图像传输、板端推理与结果校验，测量中不包含 *GUI* 展示等待。另由 *6* 次实板采集记录统计内部处理耗时，结果见表 *3-3*。

表 *3-3* 最终版实板处理与主机请求性能

项目	实测结果
PS 输入预处理	中位数 *5.65 ms*
计算图执行	中位数 *257.61 ms*
检测解码	中位数 *2.22 ms*
板端推理	中位数 *265.48 ms*
主机请求 *P50*	*362.64 ms*
主机请求 *P95*	*367.81 ms*
批次吞吐率	*2.68 FPS*

3.3.5 完成情况与综合验证

项目已完成九类缺陷模型训练与量化、*PL* 加速核心、*PS* 整网推理及 *PC* 可视化界面，形成从图像输入、板端检测到结果展示与保存的完整流程。现有证据包括离线 *800* 图验证、有限实板原始输出比对、连续同图请求和界面集成记录、长时间多图运行、独立生产批次泛化、完整数据集实板精度。

3.4 优化过程与前后对比

3.4.1 基线选择与瓶颈定位

选择首次整网实板推理版作为基线。其板端函数耗时 *6579.095 ms*，图计算占 *6403.554 ms*（*97.3%*），因此优先优化计算并行度与特征访问，再改进 *PS* 数据整理和后处理。

3.4.2 首次跑通基线与最终版对照

硬件侧由 *128* 个 *DSP* 的初版扩展为 *S12-L32-4tap* 架构，并行处理 *12* 个空间位置、*32* 路输出通道及 *4* 个卷积 *tap*，配套特征缓存、窗口生成、整数重定标和 *ACP* 访问。软件侧采用 *NEON* 输入处理、视图管理、*INT8* 分类预筛及 *DFL* 解码和 *NMS*。表 *3-4* 列出耗时和资源代价；各项修改的独立收益尚未分别测量。

表 3-4 首次跑通基线与最终版的分项耗时和资源

项目	首次跑通基线	最终性能归档版
模型与输入	*epoch108；640×640*	*epoch200；640×640*
输入预处理	27.659 ms	*5.647 ms*
图计算	6403.554 ms	*257.609 ms*
解码与 NMS	131.643 ms	2.217 ms
完整板端处理	*6579.095 ms*	*infer 265.479 ms*
主机请求 P50 / P95	*673.734 / 681.174 ms*	*362.644 / 367.810 ms*
同图 30 请求批次吞吐	0.180FPS	2.676 FPS
LUT / 寄存器	6948 / 12820	95260 / 98764
BRAM36 / DSP	70 / 128	287.5 / 1252

3.4.3 同一设计的物理优化与时序收敛

在形成上述 *12* 路实现后，还需消除布线后的建立时序违例。以同一布线后检查点为基线，保持 *RTL* 和 *10 ns* 时钟约束不变，仅执行 *Vivado* 物理优化，结果见表 *3-5*。最差建立裕量改善 *0.100 ns*，*41* 个建立时序失败端点全部消除，使该实现达到 *100 MHz* 的静态时序要求。

表 3-5 同一布线后设计的物理优化结果

项目	物理优化前	物理优化后
建立裕量 WNS / TNS	*−0.095 / −1.501 ns*	*+0.005 / 0 ns*
建立时序失败端点	41 / 314636	*0 / 314636*
保持裕量 WHS	+0.045 ns	*+0.045 ns*
LUT	*95252*	*95260*
寄存器 / BRAM36 / DSP	*98764 / 287.5 / 1252*	*98764 / 287.5 / 1252*

最终仅用增加 *8* 个 *LUT* 为代价，约占优化前 *LUT* 用量的 *0.0084%*，寄存器、*BRAM* 和 *DSP* 用量不变。最终 *WNS* 为*+0.005 ns*。

3.4.4 未直接采用的方案与经验

曾探索将部分 *33* 位反馈累加迁至逻辑资源，减少 *DSP* 占用。仅完成候选编写和主机检查，*RTL* 仿真、综合时序，布局布线难度极大，故未纳入最终实现。软件侧也放弃了未减少 *ARM* 指令的 *1×1* 打包改写，具体取舍见 *4.2.3*。

第四部分　大模型协作与技能包提炼

4.1 协作分工与工作流程

在开发过程中，我负责确定目标、限定实验范围、选择模型与部署版本，并决定哪些方案进入最终工程。大模型主要协助拆解问题、解释日志、提出候选实现、编写检查脚本和整理材料。对性能与正确性的判断，以保存的输入、输出、报告和复测结果为依据。

协作按“提出问题—整理证据—生成候选—执行检查—人类取舍—保存结果”推进。适合独立处理的材料核对、脚本修改和报告整理可以分开进行，再由主任务整合；涉及同一硬件运行或同一文件的修改，先明确执行顺序。未通过验证的候选保留在实验记录中，不直接替换成品。

4.2 典型协作记录与纠错过程

以下提示词和回答短句摘自工程包的协作索引；其余文字为过程摘要。案例覆盖早期双任务探索和后期 *PIN* 九类方案。

4.2.1 先纠正目标模型，再讨论部署成功

提示词摘录：“现在先把目前效果最好的模型的具体 ap 值告诉我，我来判断是否进入下一阶段”。当助手选择的候选与我的预期不一致时，我进一步指出：“目前效果最好的应该是两个 ap 值 *77* 左右的模型”。

回答摘录：“我会按这个目标查找对应权重和完整评估记录，先不部署刚才提到的 *76%* / *66%*版本。”该阶段随后重新核对候选权重和评估记录。助手虽然报告过 *JTAG* 下结果返回一致，但同时发现量化质量未达到当时设定的门槛，*SD* 部署也未完成。

采纳与纠正：采纳重新核对权重、指标和部署文件的做法；不把“已经输出检测框”理解成模型质量、运行身份和冷启动全部合格。

4.2.2 根据收益决定训练何时收尾

提示词摘录：“目前训练的提升效果接近饱和，我认为没有继续的必要了，还需要跑满 300 轮吗？”。回答摘录：“早期提升明显，现在已经进入收益较小的阶段；我认为没有很强的必要一定跑满 *300* 轮。”

我的决定：“可以 按照 *200* 轮收尾的想法来”。执行计划保留原学习率计划和完整断点，到 *200* 轮后收尾，再核对最佳权重和验证记录。后续记录确认 *1—200* 轮连续，没有继续完成第 *201* 轮；训练审查案例也核对了轮次完整性及 *200* 轮内既定选模指标的最大值。

采纳与边界：采纳结合验证收益安排收尾，避免仅为增加轮次而继续训练，停止训练与质量达标分别判断。

4.2.3 放弃没有收益的循环改写

提示词摘录：“简单改写循环没有降低 arm 指令，换 3*3 卷积的算法”。此前助手建议先检查软件数据搬运开销，再提出独立候选，输出一致后才考虑采用。

回答摘录：“简单改写 *1×1* 打包循环没有减少 *ARM* 指令，已放弃；现在改为优化 *3×3* 卷积内部区域，边缘仍走原有处理逻辑。”后续回答报告 *106* 组打包差分检查通过，并说明边界和尾块输出一致。

采纳与取舍：没有继续保留仅改变代码写法、却没有减少指令的方案；保留先做差分检查、再讨论收益的步骤。

4.2.4 诊断通过不等于原版故障消失

提示词摘录：“改用 *64* 位试一下”。ai 当时将其理解为 *64* 路输出通道并行；索引中没有用户确认这一解释的记录。这个歧义本身说明：接口位宽与计算并行路数应在修改前明确区分。

调试摘要：诊断版一批请求通过后，未插桩原版再次出现失败；加入暂停后两张图通过，助手修正判断：“暂停改变了时序，所以还不能算修复。”后续完整初始化未复发时，仍保留“具体原因尚未完全定位，*SD* 冷启动待验证”的结论。

采纳与纠正：保留诊断快照和分运行记录，要求回到原版复测；不把断点、插桩或一次不复发直接当作修复证据。由此形成的排查方法是先区分各次运行，再检查长度、结束标志、缓存、缓冲所有权及初始化条件。

4.3 *协作经验提炼的技能包*

我将协作经验整理为 *7* 项技能，随工程包放在 *skill/*目录。原有 *6* 项面向工程开发与验证，新增的通用 *PYNQ* 诊断技能单列于 *4.4* 节。原有技能用途如下：

模型与部署身份核对：核对模型、类别、输入约定以及固件和硬件版本，帮助选择配套部署文件。

数值一致性分析：比较浮点模型、整数参考、*C/RTL* 与实板输出，定位计算差异。

PS 与 *PL* 性能分析：整理预处理、数据搬运、硬件计算和后处理耗时，确定优化重点。

DMA 异常排查：检查传输长度、结束标志、缓存同步及缓冲区所有权，辅助定位通信与数据交接问题。

训练质量与收尾审查：核对数据划分、验证集选模、量化指标和完成轮次，整理训练结果。

Vivado 布线优化：分析拥塞、关键路径和实现策略，辅助完成布局布线与时序收敛。

4.4 通用 PYNQ 诊断技能

4.4.1 用途与通用设计

在原有工程技能基础上，我们进一步整理了可独立使用的 *PYNQ Overlay* 与 *DMA* 诊断技能 *pynq-overlay-diagnostics*。它面向部署前检查和故障初查，帮助使用者回答硬件文件是否对应目标器件、所需 *IP* 是否存在、*DMA* 能否承载本次数据，以及还缺哪些验证信息。核心脚本不包含本队检测类别、模型权重、私有通信协议或固定板卡地址。

技能从 *bit*、*HWH* 或 *XSA* 读取器件与 *IP* 信息，依据用户配置计算数据形状和类型对应的字节数，检查传输长度、缓冲容量、偏移和地址对齐；对 *notebook* 只读取代码，标出可能加载硬件的 *Overlay* 调用和缺少等待边界的 *DMA* 操作。板端环境采集使用 *download=False*，读取 *PYNQ* 版本及 *Overlay* 元数据，不执行硬件下载或 *DMA* 传输。

4.4.2 换题目与换板卡的复用验证

换题目时，主要修改输入输出的 *shape*、*dtype* 和 *IP* 配置；换板卡时，提供该板自己的 *bit/HWH* 并更新器件与 *IP* 路径，同一套诊断代码继续使用。我们使用 *7020* 与 *7100* 真实工程元数据检查平台差异，并用图像块、信号向量和公开 *PYNQ* 缩放 *notebook* 展示不同任务的复用。检查能指出超长图像超过 *DMA* 长度上限，也能识别 *7100* 原生 *ACP* 设计缺少标准 *AXI DMA*，避免套用不匹配的驱动。

目前已通过 *32* 项软件测试、*6* 项静态演示及 *2* 项独立智能体使用检查，独立压缩包解压至隔离目录后复测通过。测试覆盖传输字节数、容量与偏移、非 *DRE* 地址对齐、缺失参数、器件冲突和 *notebook* 不执行等行为。这些证据验证的是软件诊断与静态复用；本轮未在两块物理板卡上运行 *PYNQ* 推理，旧工程的原生 *C/ELF/JTAG* 结果不计作 *PYNQ* 运行成绩。

4.4.3 交付材料与评审入口

通用技能放在工程包 *skill/pynq-overlay-diagnostics* 目录，独立申报说明位于 *report/pynq_skill_submission.md*。目录内提供技能入口、标准库脚本、配置说明、公开示例及来源许可、测试和验收记录。评审者可按该目录 *readme.md* 先运行六个演示，再执行测试，无需连接板卡。缺少 *bit* 文件或实际物理地址时，报告保留未确认项，不将静态通过写成硬件运行成功。

第五部分　复现说明

本章面向首次接触本项目的复现者，按准备电脑与板卡、解压工程、安装依赖、核对输入、从 RTL 重建硬件、编译固件与启动镜像、上板检测的顺序给出操作步骤。复现使用随包冻结的 epoch200 模型与 300 张样例，目标是从交付源码重建并运行本作品。

5.1 准备工程包和工具

使用 defect_detection_z100_test300_repro_20261103.zip。解压后工程根目录名为 defect_detection_z100_repro，包含 readme.md 以及 src、sim、build、board、data、skill、report 七个目录。包内提供 RTL/ROM、PS 应用、BSP/FSBL 源码、构建脚本、冻结模型、300 对图片标签及最终上板文件；无需原开发电脑中的工程目录或历史 DCP。

硬件准备：Zynq-7Z100 开发板及配套电源、以太网线、FAT32 格式 Micro SD 卡与读卡器；调试时可接板卡串口并安装对应 USB 串口驱动。目标器件为 xc7z100ffg900-2，使用包内既定 PS、DDR、引脚及 100 MHz/ACP 配置。使用其他型号板卡时须先完成板级配置适配。

电脑安装 Windows x64 和 Python 3.12，安装 Python 时选中 pip、Tcl/Tk 及加入 PATH。安装 Vivado/Vitis 2026.1，包含 Zynq-7000 器件支持、XSim、ARM GCC 和 Bootgen；准备 CMake 4.0.1 与 MinGW Make 4.4.1。固件参考工具链为 ARM GCC 13.4.0。Vivado 启动前至少保留 8 GiB 空闲内存及输出盘 20 GiB 空闲空间。

以下命令在 PowerShell 中执行。以 C 盘短目录为例；软件安装路径按本机修改。代码中的反引号表示同一条命令续行，反引号后不要添加空格。每条命令执行成功后再继续，失败时先处理报错；重新构建应更换尚不存在的输出目录。

5.2 解压并建立 Python 环境

将压缩包放到 C:\z100_delivery，在 PowerShell 执行下列命令。解压目标应为尚未使用的目录，随后始终在含 readme.md 的工程根目录运行本章命令。

桌面依赖固定为 NumPy 2.5.4、OpenCV 5.0.0.93 和 Pillow 12.3.0；电脑整数参考另使用 PyTorch 2.14.1+cpu。依赖文件保存在 build 目录中。首次安装需要联网，之后使用同一虚拟环境执行命令。新开 PowerShell 时重新进入工程根目录，并重新设置$py 及下节的工具路径变量。

核对已安装的 Python 包和工具；各变量均指向本机实际存在的文件或目录。

5.3 核对工程文件和模型输入

在构建前执行以下输入检查。文件校验应显示 PASS_SUBMISSION_HASHES，模型为 epoch200、PIN 九类、640×640 输入；样例为 300 组图片与标签。其余检查应正常退出并显示各自的通过状态。检查不通过时，根据输出中的文件名重新解压或核对版本。

冻结板端模型为 data/pin_epoch200_int8.v8p，类别顺序见 data/model/classes.json，样例在 data/dataset/images 和 labels 中，逐图参考预测为 report/selected_sample_predictions.json。所有--check 仅核对输入；后续构建、仿真及检测结果分别在包外生成。

5.4 从 RTL 重建硬件并运行组件仿真

先从 53 份固定 RTL/ROM 及包内 BD/Tcl 重新综合、布局、布线与导出。使用短输出路径，避免 Windows 生成 IP 文件时路径过长。关闭占用同一工具锁的其他 Vivado、仿真或板卡调试任务，再执行：

脚本在 C:\z100_hw 下新建 run-*目录。完成后打开该目录的 run.json，应为 EXPORT_PASS_BOARD_PENDING；时序、路由及 DRC 检查通过后，export/artifacts 目录中应生成 z100_platform.bit 和 z100_platform.xsa。若输出目录因路径过长被拒绝，改用更短的包外目录；若资源不足或工具被占用，释放资源后以新目录重试。

记录本次成功构建的实际 run-*目录，下一节封装启动镜像时使用其中的 bit。下面用交互输入避免手写时间戳；粘贴完整目录路径即可，不含外层引号。

Test-Path 应返回 True。核对同目录的 XSA 及 run.json 属于同一次成功运行，保持既定硬件地址、时钟、DDR 和中断配置，以使用包内配套 BSP/FSBL。修改这些配置后需重新生成匹配的软件平台。

硬件构建结束并释放工具锁后，可执行 9 项 RTL 组件/核心回归。输出目录必须在包外且尚不存在

在 C:\z100_out\rtl_all\run_report.json 中检查 9 项用例均通过，以及 xvlog、xelab、xsim 的返回码和通过标志。测试台、ROM、输入及期望值均随包提供。组件回归的范围与用例说明见 sim/rtl/reproduction.md。

5.5 编译固件并生成启动镜像

上板运行采用 SD 网络修复版。以下入口从 build/bsp_snapshot 重建 BSP 和 FSBL，编译全部 90 个应用对象并加入网络修复源码，再由 Bootgen 封装启动镜像。$bit 沿用上节成功导出的文件，显式指定它可避免默认使用包内参考位流。

完成后，C:\z100_out\sd_source 应包含 fsbl.elf、native_board_pool.elf、z100_platform.bit、boot.bif、BOOT.BIN 及 result.json。result.json 应为

BOOTGEN_COMPLETE_BOARD_PENDING，并记录所选位流、固件和启动镜像的SHA256。该 BOOT.BIN 即为下一节写入 SD 卡的源码重建结果。

若结果明确为 BOOTGEN_PENDING_LOCK_BUSY 或 BOOTGEN_PENDING_TOOL_BUSY，待占用工具退出后，可在同一目录继续封装，无需重新编译：

如需另行重建用于历史性能对照的应用，使用下列入口。它生成独立 ELF，不替换上述 SD 运行镜像。

此目录的 native_board_pool.elf 为性能归档版源码重建结果，状态为 BUILT_ARM_ELF_BOARD_PENDING。历史 2.676 FPS 对应原性能归档版；新生成的 ELF 与 SD 网络修复版应各自记录运行结果。

5.6 上板启动并运行桌面检测

第一步，关闭板卡电源，使用读卡器打开 FAT32 格式 SD 卡。将 C:\z100_out\sd_source\BOOT.BIN 复制到 SD 卡分区根目录。若卡中已有启动文件，先另行备份，再替换为本次镜像。安全弹出 SD 卡并插入板卡 PS Micro SD 卡槽。

第二步，按开发板厂家手册设置 SD 启动模式；本项目原板使用记录为 BOOT_CFG 两个开关均 OFF。接好网线与配套电源，可同时接串口观察启动信息。重新上电，执行一次断电后的冷启动。

第三步，在 Windows“网络连接”中找到连接板卡的以太网适配器，将 IPv4 手动设为 192.168.10.1，子网掩码 255.255.255.0，直连时网关与 DNS 留空。板卡使用 192.168.10.2，TCP 服务端口 5001。多网卡时确认该适配器没有被其他同网段连接抢占。

第四步，回到工程根目录启动上位机；需要放行时允许 Python 通过当前网络的防火墙规则。

第五步，点击“检查连接”，确认协议、模型和类别身份核对通过。导入 data/dataset/images 中的一张图片并开始检测，界面应显示原图、检测框、九类中的类别、置信度和详情表。随后导入多张图片，检查队列顺序、暂停与继续、结果保存和历史查询。结果记录保存在 C:\z100_out\desktop_history。

连接失败时，先检查板卡启动、网线链路和电脑 IPv4 地址，再核对 5001 端口与防火墙；模型身份不符时核对所用镜像和 data 中的冻结模型。检测后保存本次 BOOT、bit 及 ELF 的身份、输入图片、返回结果和界面截图。

如需先运行随包已有成品，可用 board/sd_network_fix_v1/boot.bin 作为 SD 卡根目录的 BOOT.BIN，其他接线与界面步骤相同。源码重建结果与已有成品分别存放，便于定位环境或构建差异。

5.7 复现 300 张样例的检测输出

先关闭上位机，避免与批量入口同时连接板卡。下列命令逐张向板卡发送全部 300 张样例，并与包内电脑整数参考预测比较；默认归一化坐标及置信度绝对误差容差为 1e-3。

完成后查看 C:\z100_out\board300\result.json：backend 应为 board，board_access 为 true，images 为 300，failures 为空且 status 为 PASS_DETECTION_300。该入口核对正式板端协议及模型身份，连接异常或逐图差异超限会返回失败。

电脑整数参考可独立运行，用于核对冻结模型和参考预测。它使用前面安装的 CPU PyTorch 环境，命令如下；输出目录同样应尚不存在。

查看 reference300/result.json，全部 300 张通过时同样显示 PASS_DETECTION_300，backend 为 reference、board_access 为 false。预处理固定为 RGB、640×640 等比例居中补边、114 填充；置信度阈值 0.25、类别内 NMS IoU 0.45、最多 300 框。电脑结果用于数值对照，板端结果用于确认作品实际运行。

300 张来自原 test 的优选展示子集，图片、标签和清单随包保留。模型与量化材料见 src/model_pipeline、data/model 及 report；复现从冻结权重开始，无需重训。

5.8 保存复现结果

保存本机工具版本、硬件 run.json、固件 result.json、RTL run_report.json、检测 result.json 及界面截图。构建中间产物和日志留在包外。

已有 9 项 RTL 组件回归与 300 张电脑整数参考通过记录。新硬件构建在布线阶段停止，尚无新 bit/XSA 导出；重建产物上板与 SD 冷启动按本章另行记录。

表 5-1 从零复现各阶段的输出与完成标志

阶段	主要输出	完成标志
输入核对	文件清单与模型输入检查	文件、模型及 300 组样例检查通过。
硬件重建	run.json、bit 和 XSA	时序、路由、DRC 通过并导出。
RTL 回归	rtl_all/run_report.json	9 项组件/核心用例通过。
固件与镜像	ELF、BOOT 和构建记录	完成编译和封装，记录输入 SHA。
上板检测	冷启动及检测记录	身份正确，300 张板端比较通过。
电脑参考	reference300/result.json	300 张电脑整数参考比较通过。

第六部分　总结

6.1 可扩展之处

（1）提升硬件算力与资源使用效率。当前项目仍可探索将部分反馈累加从 *DSP* 迁至进位链逻辑，在激活、权重读取等长路径增加寄存级，再评估扩大并行阵列或提高频率。难点在于同时满足数值一致性、存储带宽、布线时序与功耗约束。

（2）提高连续检测能力并对接产线。当前项目可以通过双缓冲和多帧流水，使 *PS* 预处理、后处理与 *PL* 计算重叠，并逐步接入工业相机与产线触发。需要解决缓冲区所有权、并发同步、采集节拍匹配和异常恢复问题，保证连续任务下的结果正确性与稳定性。

（3）拓展器件类型与复杂检测场景。针对不同材质、光照和缺陷尺寸补充数据，开展跨批次训练、*INT8* 校准与实板验证，逐步扩展检测范围。难点是控制少样本和场景变化带来的漏检，同时兼顾微小缺陷精度及板端计算、存储开销。

（4）建设可复用的部署与交互平台。可探索 *PYNQ Overlay* 封装及 *Python* 接口，将模型加载、硬件调用与可视化管理统一起来。需要处理操作系统与驱动适配、算子覆盖、量化一致性和版本兼容，使不同模型或板卡的迁移具备可验证的流程。

6.2 心得体会

先明确任务，再让模型协助推进。回顾比赛历程，从梳理赛题、板卡和软硬件分工，到训练、量化、上板与界面联调，我主要让大模型协助拆解问题、整理资料、编写候选代码和分析日志。方案可以由模型提出，但目标、优先级和验收条件需要我持续明确。

训练与部署要分别判断。训练阶段，我结合验证指标采纳了在 *200* 轮收尾的建议，没有把增加轮次当作必然有效的办法。部署过程中，我曾纠正待部署模型的选择，也逐渐认识到，检测头一致只能证明执行一致，不能代替完整验证集上的量化精度评价，所以在模型的最后训练效果趋于饱和后，我选择停止并进行下一阶段的任务。

建议是否采用，要看实际收益。优化阶段，我采纳了先测分项耗时、再调整软硬件分工的路线，将注意力从单纯增加计算单元转向数据搬运和处理开销。对于没有减少 *ARM* 指令的 *1×1* 打包改写，协作中没有继续保留；并行规模和时序修改也要经过比较后再取舍。

调试成功必须经得起复测。模型帮助分析过 *DMA* 异常、数值差异和时序问题，也出现过过早判断修改有效的情况。诊断版或断点后通过，不能直接说明原版已修复。我逐渐形成了保留基线、分层比较、原版复测和连续检测的习惯，不因一次通过就接受结论。

开发顺序要服务于完整应用。中期我曾要求暂停界面打磨，优先接入新模型并验证链路；后期再反复要求补齐连接检查、批量检测、原图对照和结果导出。这让我体会到，底层计算正确与用户操作方便都要考虑，但应根据阶段安排先后。

协作的价值在于帮助我作出判断。大模型提高了查资料、试方案和整理工程的效率，我需要追问建议依据、比较已采用与未采用方案，并把实测结果落实到文档。最终保留清楚的基线和成品版本，整理可复用流程，也是这次开发的重要收获。

第七部分　参考文献

[1] *Zhao X, Lyu Q, Zeng H, et al. A defect detection algorithm for optoelectronic detectors utilizing GLV-YOLO*[J]. *Micromachines, 2025, 16*(3): *267*.
[2] *Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8*[CP/OL]. (*2023*)[*2026-10-29*]. *https://github.com/ultralytics/ultralytics*.
[3] *Redmon J, Divvala S, Girshick R, et al. You only look once: unified, real-time object detection*[C]//*Proceedings of CVPR. 2016: 779-788*.
[4] *Li X, Wang W, Wu L, et al. Generalized focal loss: learning qualified and distributed bounding boxes for dense object detection*[C]//*Advances in NeurIPS. 2020: 21002-21012*.
[5] *Jacob B, Kligys S, Chen B, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference*[C]//*Proceedings of CVPR. 2018: 2704-2713*.
[6] *Umuroglu Y, Fraser N J, Gambardella G, et al. FINN: a framework for fast, scalable binarized neural network inference*[C]//*Proceedings of FPGA. 2017: 65-74*.
[7] *Qiu J, Wang J, Yao S, et al. Going deeper with embedded FPGA platform for convolutional neural network*[C]//*Proceedings of FPGA. 2016: 26-35*.
[8] *Zhang C, Li P, Sun G, et al. Optimizing FPGA-based accelerator design for deep convolutional neural networks*[C]//*Proceedings of FPGA. 2015: 161-170*.
[9] *Fu Y, Wu E, Sirasao A, et al. Deep learning with INT8 optimization on Xilinx devices*[R]. *Xilinx White Paper WP486, 2017*.
[10] *AMD. Zynq 7000 SoC technical reference manual* (*UG585*)[Z].
[11] *AMD. 7 series DSP slice user guide* (*UG479*)[Z].
[12] *AMD. UltraFast design methodology guide for FPGAs and SoCs* (*UG949*)[Z].
[13] *AMD. Vivado Design Suite user guide: design analysis and closure techniques* (*UG906*)[Z].